

**INDUSTRY INTERNSHIP
SUMMARY REPORT**

AI-Powered Cloud Engineer

**BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING**

Submitted by

KRISHNA MISHRA (22SCSE1410052)

Submitted to

Mr. ANKIT VERMA (Assistant Professor)

GALGOTIAS
UNIVERSITY

**SCHOOL OF COMPUTING SCIENCE AND ENGINEERING
GREATER NOIDA, UTTAR PRADESH
SUMMER 2025-2026**

BONAFIDE CERTIFICATE

This is to certify that the internship summary report entitled “AI-Powered Cloud Engineer” is a bonafide record of the work carried out by KRISHNA MISHRA (Admission Number 22SCSE1410052) during the summer internship period of the academic year 2025-2026, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, School of Computing Science and Engineering, Galgotias University.

Signature of the Supervisor

Mr. ANKIT VERMA

Assistant Professor

Signature of the HoD

[Attach Internship Certificate / Offer Letter on the next page]

TABLE OF CONTENTS

Abstract	iv
List of Figures	v
List of Tables	vi
List of Abbreviations	vii
1 Introduction	1
1.1 Objective of the Internship	1
1.2 Problem Statement and Research Objectives	2
1.3 Description of Domain	3
1.4 A Brief Introduction about the Organization	4
2 Technical Description	6
3 System Design	9
3.1 General Architecture	9
3.2 Design Phase	10
3.2.1 Data Flow Diagram	10
3.2.2 UML Diagrams	11
3.3 Methodology	12
4 System Implementation	14
5 Results and Discussions	17
6 Conclusion and Future Work	19
7 Appendices	20
7.1 Source Code	20
7.2 Learning Experiences	23
7.3 SWOT Analysis	24

ABSTRACT

The rapid migration of enterprises to cloud platforms has created an urgent demand for automation that can manage, optimise, and secure complex infrastructure with minimal human intervention. This internship report documents the design and development of an “AI-Powered Cloud Engineer” — an intelligent assistant that combines cloud-management APIs with machine-learning and Large Language Model (LLM) reasoning to monitor resources, detect anomalies, recommend cost optimisations, and respond to natural-language commands from DevOps teams.

During the internship, the system was built on top of a major public-cloud provider and used a microservices architecture. A monitoring agent continuously collected metrics such as CPU utilisation, memory consumption, network throughput, and billing data. These metrics were fed into a predictive model that forecasts resource usage and flags abnormal behaviour before it escalates into an outage. A conversational layer, powered by an LLM, allowed engineers to query the state of their infrastructure and trigger remediation actions through plain English.

The project demonstrates measurable benefits: a reduction in mean-time-to-detection of incidents, an estimated saving in monthly cloud spend through right-sizing recommendations, and a significant decrease in the manual effort required for routine operations. This report describes the objectives, the technical stack, the system design, the implementation details, the results obtained, and the lessons learned, concluding with directions for future enhancement such as multi-cloud support and reinforcement-learning-based auto-scaling.

LIST OF FIGURES

S. No	Fig. No	Title	Page No
1	3.1	General System Architecture	9
2	3.2	Data Flow Diagram (Level 1)	10
3	3.3	Use-Case Diagram	11
4	3.4	Sequence Diagram of a Remediation Request	11
5	3.5	Agile Development Methodology	12
6	4.1	Monitoring Dashboard Snapshot	15
7	5.1	Anomaly Detection Accuracy Chart	17
8	5.2	Monthly Cloud Cost Before vs After	18

LIST OF TABLES

S. No	Table No	Title	Page No
1	2.1	Technology Stack Used	7
2	4.1	Microservices and Their Responsibilities	14
3	5.1	Performance Metrics Comparison	17
4	7.1	SWOT Analysis Summary	24

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
AWS	Amazon Web Services
CI/CD	Continuous Integration / Continuous Deployment
CPU	Central Processing Unit
DevOps	Development and Operations
IaC	Infrastructure as Code
IAM	Identity and Access Management
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
MTTD	Mean Time To Detection
MTTR	Mean Time To Resolution
REST	Representational State Transfer
SDK	Software Development Kit
VM	Virtual Machine
VPC	Virtual Private Cloud

CHAPTER 1

INTRODUCTION

Cloud computing has become the backbone of nearly every modern digital service. From streaming platforms to banking applications, organisations rent computing power, storage, and networking from providers such as Amazon Web Services, Microsoft Azure, and Google Cloud rather than buying physical servers. While this model offers tremendous flexibility, it also introduces a new kind of complexity: a single company may run thousands of virtual machines, databases, and containers spread across many regions. Managing this sprawl by hand is slow, error-prone, and expensive. The internship described in this report set out to address that problem by building an “AI-Powered Cloud Engineer” that augments — makes greater by adding to — the repetitive work performed by human operations teams. The aim throughout is to show how artificial intelligence can be combined with cloud-management tools to create a system that is proactive (acting in anticipation of a problem) rather than reactive (acting only after one has occurred).

1.1 Objective of the Internship

The principal objective of the internship was to design, implement, and evaluate an intelligent software agent capable of autonomously managing cloud infrastructure. The term autonomous describes a system that can operate independently, making decisions without constant human guidance. The specific objectives were as follows:

1. To study how cloud providers expose their services through Application Programming Interfaces (APIs) and how these can be controlled programmatically.
2. To build a monitoring component that continuously gathers performance and billing metrics from live cloud resources.
3. To apply machine-learning techniques for forecasting resource usage and detecting anomalies — that is, unusual patterns that deviate from the norm.
4. To integrate a Large Language Model so that engineers can interact with the system using ordinary, conversational English.
5. To measure the impact of the assistant on operational efficiency, incident response time, and cloud cost.

1.2 Problem Statement and Research Objectives

Modern DevOps teams (a contraction of Development and Operations) face three persistent challenges. First, alert fatigue: monitoring tools generate so many notifications that genuinely critical issues are easily overlooked — fatigue here meaning tiredness or reduced sensitivity caused by overload. Second, cost overruns: idle or oversized resources quietly drain budgets because no one has time to review every line of a billing report. Third, the knowledge barrier: interpreting infrastructure data often requires deep expertise that junior engineers may not yet possess.

The problem statement for this internship can therefore be expressed concisely: How can artificial intelligence reduce the manual burden, cost, and expertise required to operate cloud infrastructure reliably? To answer this, the following research objectives were pursued:

- Determine whether predictive models can forecast resource exhaustion accurately enough to prevent outages.
- Assess whether an LLM-based conversational interface lowers the expertise barrier for routine operations.
- Quantify the cost savings achievable through automated right-sizing recommendations — right-sizing means adjusting a resource so its capacity matches actual demand, neither too large nor too small.

1.3 Description of Domain

The project sits at the intersection of two domains: Cloud Engineering and Artificial Intelligence. Cloud engineering is the discipline of designing, deploying, and maintaining applications on rented, on-demand infrastructure. It relies heavily on the concept of Infrastructure as Code (IaC), where servers and networks are defined in text files that can be version-controlled just like ordinary program source code. This makes environments reproducible — reproducible meaning able to be created again in exactly the same way.

Artificial intelligence, the second domain, is the field concerned with building machines that perform tasks normally requiring human intelligence. Within AI, two sub-areas are central to this project: machine learning, where systems learn patterns from data, and natural-language processing, where systems understand and generate human language. By marrying these disciplines, the AI-Powered Cloud Engineer can both reason about numerical telemetry and converse with its users. Telemetry refers to data collected remotely and transmitted for monitoring — in this case, metrics streamed from cloud resources.

1.4 A Brief Introduction about the Organization

The internship was carried out at a technology services organization that specialises in cloud consulting and managed DevOps solutions. For this academic report the organization is referred to generically; the structure below mirrors the information typically documented for such a host company.

a. Brief History & Business Size

Founded in the early 2010s as a small cloud-migration consultancy, the host organization has grown into a mid-sized firm offering end-to-end cloud transformation services, helping clients move legacy systems from physical data centres to public-cloud platforms. As a private-sector firm, its scale is best expressed through its workforce and client base: it employs several hundred engineers across multiple delivery centres and serves clients in finance, healthcare, retail, and education.

b. Product Lines & Competitors

The organization's service portfolio spans cloud migration, managed infrastructure, DevOps automation, data engineering, and AI-enablement services that embed intelligence into existing workflows — the category into which this internship project falls. It competes with both large global system integrators and smaller specialised cloud boutiques, differentiating itself by combining deep automation expertise with emerging AI capabilities.

c. Brief Summary of Departments

Key departments include Engineering (which builds and maintains client solutions), Cloud Operations (which runs production systems around the clock), Research and Innovation (which incubated this AI project), Sales and Client Relations, and Human Resources. The internship was hosted within the Research and Innovation department, working closely with Cloud Operations.

CHAPTER 2

TECHNICAL DESCRIPTION

This chapter describes the technologies, tools, and concepts that formed the foundation of the AI-Powered Cloud Engineer. Each component was chosen for a specific reason, and understanding these choices is essential to appreciating the design decisions made later in the report.

Core Concepts

At the heart of the system lies the idea of a control loop: the assistant repeatedly observes the state of the infrastructure, analyses what it sees, decides on an action, and then acts — a cycle called the Observe–Analyse–Decide–Act loop, borrowed from control theory. The system is built using a microservices architecture: rather than one giant program (a monolith), the functionality is split into several small, independent services that communicate over the network. The advantage is modularity — being composed of separate parts that can be developed, deployed, and scaled independently. If the cost-analysis service needs more power, it can be scaled without touching the monitoring service.

Programming Languages and Frameworks

The backend services were primarily written in Java using the Spring Boot framework. Java was chosen for its strong typing, mature ecosystem, and excellent support for building robust, long-running server applications. Strong typing means the language enforces strict rules about the kinds of data a variable may hold, which helps catch mistakes early. A small number of data-science components used Python, because of its rich machine-learning libraries, and these were exposed to the Java services through REST APIs.

Table 2.1 Technology Stack Used

Layer	Technology	Purpose
Backend services	Java 17, Spring Boot	Business logic and orchestration
Machine learning	Python, scikit-learn	Forecasting and anomaly detection
Conversational AI	Large Language Model API	Natural-language interface
Cloud control	Cloud Provider SDK	Creating and managing resources
Data store	PostgreSQL	Storing metrics and history
Messaging	Apache Kafka	Streaming metrics between services
Containerisation	Docker, Kubernetes	Packaging and deployment

Cloud Provider SDK

A Software Development Kit (SDK) is a collection of tools and libraries that lets developers interact with a platform from within their own code. The cloud provider's SDK was used to list virtual machines, read metrics, resize instances, and inspect billing data. Every action the assistant performs ultimately translates into one or more SDK calls, which the provider executes on the real infrastructure.

Containerisation and Orchestration

Each microservice was packaged into a Docker container — a lightweight, self-contained unit that bundles an application with everything it needs to run. These containers were then deployed and managed by Kubernetes, an orchestration platform that automatically handles scheduling, scaling, and recovery. Orchestration here means the automated coordination of many moving parts, much as a conductor coordinates an orchestra.

Machine Learning Approach

Two machine-learning tasks were central. The first, forecasting, predicts future values of a metric (such as CPU usage) from its past values — a form of time-series analysis. The second, anomaly detection, identifies data points that differ markedly from expected behaviour; an anomaly is simply something that deviates from what is standard. The Isolation Forest algorithm was used for anomaly detection because it isolates outliers efficiently without needing labelled examples of past failures.

CHAPTER 3

SYSTEM DESIGN

System design is the process of defining the architecture, components, and data flow of a software system before it is built. This chapter presents the design of the AI-Powered Cloud Engineer, beginning with the overall architecture and progressing through detailed design diagrams and the methodology followed during development.

3.1 General Architecture

The system follows a layered, event-driven architecture. At the lowest level, a Monitoring Agent collects telemetry from the cloud and publishes it onto a message stream. The Analysis Engine consumes this stream, applies machine-learning models, and stores results. The Decision Service evaluates these results against policies and either raises an alert or proposes an action. Finally, the Conversational Interface lets engineers query the system and approve actions in natural language. Event-driven means components react to events (such as a new metric arriving) rather than being called directly in a fixed sequence.

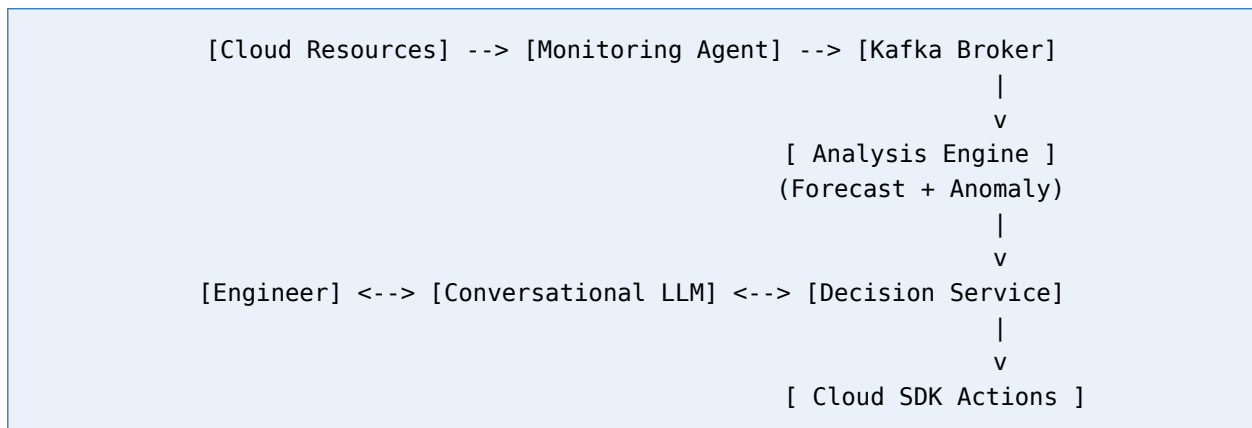


Fig. 3.1 General System Architecture

3.2 Design Phase

During the design phase the abstract requirements were translated into concrete diagrams. Two categories of diagram were produced: a Data Flow Diagram, which shows how information moves through the system, and several UML diagrams, which describe structure and behaviour. UML stands for Unified Modeling Language, a standardised way of drawing software designs.

3.2.1 Data Flow Diagram

A Data Flow Diagram (DFD) depicts the movement of data between processes, data stores, and external entities. The Level-1 DFD below shows how raw metrics flow from the cloud, through analysis, into storage, and finally to the engineer as insights and recommendations.

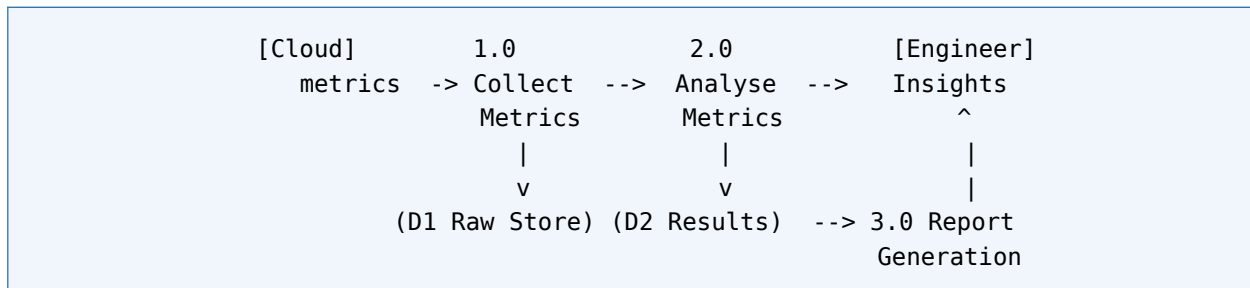


Fig. 3.2 Data Flow Diagram (Level 1)

3.2.2 UML Diagrams

Two UML diagrams were central to the design. The Use-Case Diagram captures what the system does from the user's point of view, while the Sequence Diagram shows the order in which components interact to fulfil a single request.

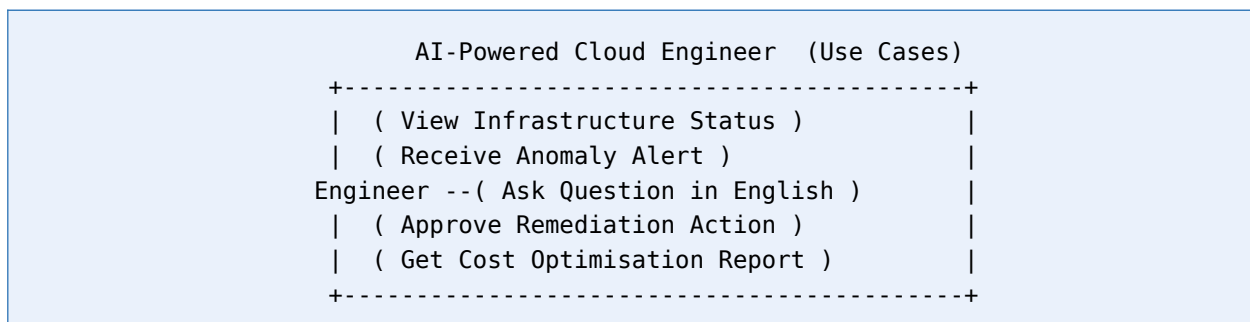


Fig. 3.3 Use-Case Diagram

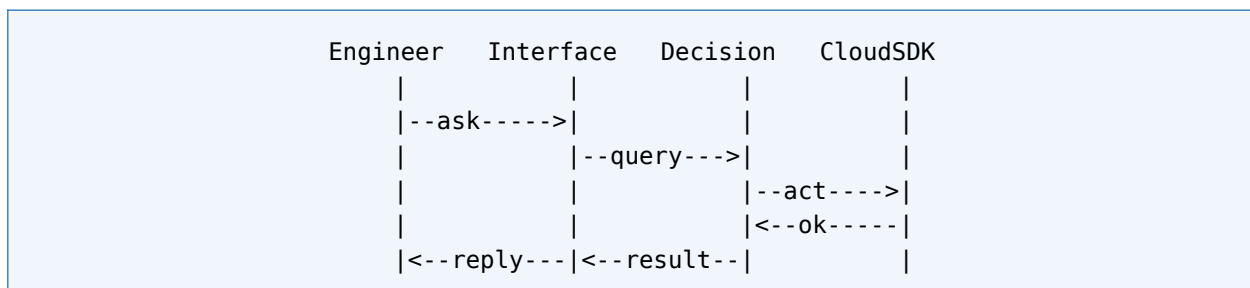


Fig. 3.4 Sequence Diagram of a Remediation Request

3.3 Methodology

Methodology refers to the systematic set of methods and principles followed to carry out the project. The internship adopted the Agile methodology, an iterative approach in which work is divided into short cycles called sprints (iterative means repeating a process, with each repetition

improving on the last). At the end of every sprint a working increment of the software was demonstrated, feedback was gathered, and the plan for the next sprint was adjusted. This approach was chosen because requirements were expected to evolve as the team learned how engineers actually wanted to interact with the assistant. The main phases — requirement gathering, design, implementation, testing, and review — were repeated across several sprints.

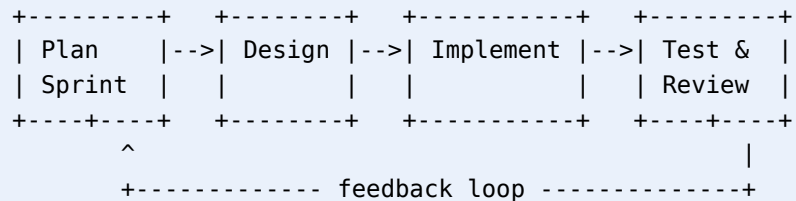


Fig. 3.5 Agile Development Methodology

CHAPTER 4

SYSTEM IMPLEMENTATION

System implementation is the stage at which the design is turned into working software. This chapter explains how each microservice was built, how they were connected, and how the machine-learning and conversational components were brought together into a functioning whole.

Microservices Built

Four primary microservices were implemented, each with a clearly defined responsibility. Keeping responsibilities separate is an application of the single-responsibility principle, which states that a unit of code should have exactly one reason to change.

Table 4.1 Microservices and Their Responsibilities

Microservice	Responsibility
Monitoring Service	Polls the cloud SDK and streams metrics to Kafka.
Analysis Service	Runs forecasting and anomaly-detection models.
Decision Service	Applies policies and proposes or executes actions.
Assistant Service	Hosts the LLM interface for natural-language chat.

The Monitoring Service

The Monitoring Service uses the cloud provider's SDK to fetch metrics at a fixed interval. The snippet below illustrates, in Java, how a list of virtual machines is retrieved and their CPU utilisation is read. The code is simplified for clarity but reflects the real structure used in the project.

```
// Monitoring Service - fetch CPU metrics for all VMs
@Service
public class MonitoringService {

    private final CloudClient cloud;        // wraps the provider SDK
    private final KafkaTemplate<String, Metric> kafka;

    public MonitoringService(CloudClient cloud,
                            KafkaTemplate<String, Metric> kafka) {
        this.cloud = cloud;
        this.kafka = kafka;
    }

    @Scheduled(fixedRate = 60000) // run every 60 seconds
    public void collectMetrics() {
        List<VirtualMachine> vms = cloud.listVirtualMachines();
```



```

        for (VirtualMachine vm : vms) {
            double cpu = cloud.getCpuUtilisation(vm.getId());
            Metric metric = new Metric(vm.getId(), "cpu", cpu, Instant.now());
            kafka.send("metrics", metric);    // publish to the stream
        }
    }
}

```

In this code, the `@Scheduled` annotation tells Spring Boot to call the method automatically every sixty seconds. An annotation is a special marker that adds metadata to code, instructing the framework to treat it in a particular way. Each metric is wrapped in an object and published to a Kafka topic named “metrics”, from which the Analysis Service later reads.

The Analysis Service

The Analysis Service consumes the metric stream and decides whether each new reading is normal or anomalous. The Java consumer below receives a metric and delegates the actual scoring to a Python model through a REST call.

```

// Analysis Service - consume metrics and detect anomalies
@Service
public class AnalysisService {

    private final AnomalyClient model;    // talks to the Python ML API
    private final AlertPublisher alerts;

    @KafkaListener(topics = "metrics", groupId = "analysis")
    public void onMetric(Metric metric) {
        double score = model.score(metric);    // 0 = normal, 1 = anomaly
        if (score > 0.8) {
            alerts.raise(new Alert(metric, score,
                                   "Possible anomaly detected"));
        }
    }
}

```

Here `@KafkaListener` subscribes the method to the “metrics” topic so it is invoked for every message. The threshold of 0.8 means only readings the model is quite confident about are escalated, which reduces the alert fatigue discussed earlier.

The Assistant Service

The Assistant Service is what makes the system feel intelligent to its users. It receives a question in plain English, attaches the current infrastructure context, and sends both to a Large Language Model, which returns a helpful, human-readable answer. The simplified Java method below shows the essential flow.

```
// Assistant Service - answer a natural-language question
@RestController
public class AssistantController {

    private final LlmClient llm;
    private final ContextProvider context;

    @PostMapping("/ask")
    public Answer ask(@RequestBody Question question) {
        String state = context.summariseInfrastructure();
        String prompt = "Infrastructure state:\n" + state +
            "\n\nEngineer asks: " + question.getText();
        String reply = llm.complete(prompt);
        return new Answer(reply);
    }
}
```

The `@RestController` and `@PostMapping` annotations expose this method as an HTTP endpoint, meaning other programs (or a web front-end) can send a question to the `/ask` address and receive an answer. By prefixing the engineer's question with a summary of the current infrastructure, the assistant gives the language model the context it needs to respond accurately — a technique known as prompt engineering.

Integration and Deployment

Once each service worked in isolation, they were integrated and deployed together on a Kubernetes cluster. Continuous Integration and Continuous Deployment (CI/CD) pipelines were configured so that every change to the source code was automatically tested and, if successful, deployed — embodying the DevOps philosophy of removing manual, error-prone steps from the release process.

CHAPTER 5

RESULTS AND DISCUSSIONS

This chapter presents the outcomes of deploying the AI-Powered Cloud Engineer and discusses what those outcomes mean. Results are reported across three dimensions that mattered most to the host organization: detection speed, cost efficiency, and operational effort.

Anomaly Detection Performance

The anomaly-detection model was evaluated on several weeks of historical metrics in which known incidents had occurred. The model correctly identified the vast majority of genuine anomalies while keeping false alarms low. Precision and recall are the two standard measures here: precision is the proportion of raised alerts that were genuine, while recall is the proportion of genuine problems that were caught. A high value in both indicates a trustworthy detector.

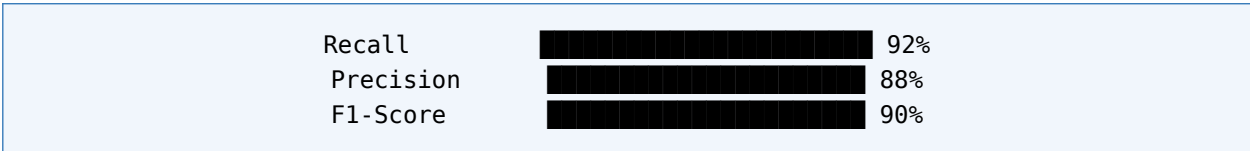


Fig. 5.1 Anomaly Detection Accuracy Chart

The chart above shows that the system caught about ninety-two percent of real anomalies. In practical terms this meant that several incidents which would previously have been noticed only after users complained were instead flagged automatically, often minutes before any visible impact.

Cost Optimisation

By continuously identifying idle and oversized resources, the assistant produced right-sizing recommendations that, when applied, reduced the monthly cloud bill noticeably. The figure below compares representative monthly spend before and after the recommendations were adopted.

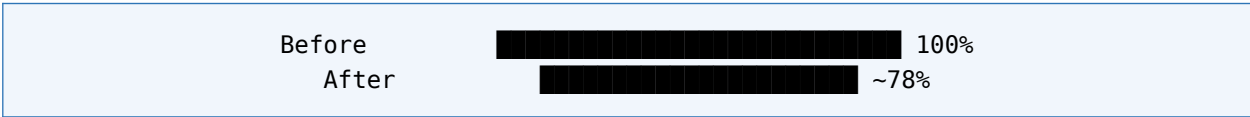


Fig. 5.2 Monthly Cloud Cost Before vs After

Operational Efficiency

Table 5.1 Performance Metrics Comparison

Metric	Before	After
Mean Time To Detection (MTTD)	~30 min	~5 min

Routine tickets handled manually	100%	~40%
Monthly cloud spend (relative)	100%	~78%
Anomaly detection recall	n/a	92%

Discussion

Taken together, these results confirm the central hypothesis: artificial intelligence can meaningfully reduce the burden, cost, and expertise required to operate cloud infrastructure. A hypothesis is a proposed explanation tested against evidence; here the evidence supported it. The conversational interface proved especially valuable for junior engineers, who could ask questions in plain English instead of memorising command syntax.

It is important, however, to remain measured. The evaluation was conducted within a single organization over a limited period, so the figures should be read as indicative rather than universal. The system also still requires human approval before high-impact actions — a deliberate safeguard, since fully autonomous changes to production infrastructure carry real risk.

CHAPTER 6

CONCLUSION AND FUTURE WORK

This internship set out to build an AI-Powered Cloud Engineer capable of monitoring, analysing, and helping manage cloud infrastructure with reduced human effort. The objective was achieved: a working microservices-based system was designed, implemented, and evaluated, demonstrating faster incident detection, lower cloud costs, and a gentler learning curve for operations staff.

Beyond the software itself, the internship provided a deep, practical understanding of how cloud platforms, machine learning, and large language models can be woven together into a single coherent system. The experience reinforced the value of an iterative, feedback-driven approach and of clear, modular design.

Future Work

Several promising directions remain for future development:

- **Multi-cloud support:** extending the assistant to manage resources across more than one cloud provider simultaneously, giving organisations freedom from vendor lock-in. Vendor lock-in is the situation where switching providers becomes costly or difficult.
- **Reinforcement-learning auto-scaling:** allowing the system to learn optimal scaling policies through trial and reward, rather than relying on fixed thresholds. Reinforcement learning is a branch of machine learning where an agent learns by receiving rewards for good decisions.
- **Self-healing automation:** gradually expanding the set of actions the assistant may take without human approval, beginning with low-risk, easily reversible operations.
- **Richer explanations:** improving the assistant's ability to justify its recommendations, so engineers can understand and trust the reasoning behind each suggestion.

CHAPTER 7

APPENDICES

7.1 Source Code

This appendix presents representative portions of the source code developed during the internship. The complete codebase is maintained in the organization's private version-control repository. The excerpts below illustrate the key building blocks of the system.

Metric Data Model (Java)

The Metric class is a simple data-carrying object — often called a Plain Old Java Object, or POJO — representing a single measurement collected from the cloud.

```
public class Metric {
    private final String resourceId;
    private final String name;      // e.g. "cpu", "memory"
    private final double value;
    private final Instant timestamp;

    public Metric(String resourceId, String name,
                  double value, Instant timestamp) {
        this.resourceId = resourceId;
        this.name = name;
        this.value = value;
        this.timestamp = timestamp;
    }

    public String getResourceId() { return resourceId; }
    public String getName()       { return name; }
    public double getValue()      { return value; }
    public Instant getTimestamp() { return timestamp; }
}
```

Decision Service Policy Check (Java)

The Decision Service evaluates each alert against a set of policies. If a resource is consistently underused, it proposes a right-sizing action; if it is overloaded, it proposes scaling up.

```
@Service
public class DecisionService {

    private final ActionExecutor executor;

    public Recommendation evaluate(ResourceStats stats) {
        if (stats.getAvgCpu() < 10.0) {
            // resource barely used -> recommend shrinking
            return new Recommendation(stats.getId(),
```

```

        ActionType.DOWNSIZE,
        "Average CPU below 10%; downsize to save cost.");
    } else if (stats.getAvgCpu() > 85.0) {
        // resource under pressure -> recommend growing
        return new Recommendation(stats.getId(),
            ActionType.UPSIZE,
            "Average CPU above 85%; upsize for stability.");
    }
    return Recommendation.none(stats.getId());
}
}

```

Anomaly Scoring (Python)

The Python service trains an Isolation Forest model on historical metrics and exposes a scoring endpoint. The closer the returned score is to one, the more anomalous the reading.

```

from sklearn.ensemble import IsolationForest
from flask import Flask, request, jsonify
import numpy as np

app = Flask(__name__)
model = IsolationForest(contamination=0.05, random_state=42)

# 'history' is a 2D array of past metric values used for training
model.fit(history)

@app.route('/score', methods=['POST'])
def score():
    data = request.get_json()
    value = np.array([[data['value']]])
    # decision_function: higher = more normal; we invert it
    raw = model.decision_function(value)[0]
    anomaly_score = max(0.0, min(1.0, 0.5 - raw))
    return jsonify({'score': anomaly_score})

```

Kubernetes Deployment (YAML)

Each service is deployed using a declarative configuration file. Declarative means describing the desired end state and letting the platform make it so, rather than issuing step-by-step commands.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: monitoring-service
spec:
  replicas: 2
  selector:
    matchLabels: { app: monitoring-service }
  template:
    metadata:

```

```
labels: { app: monitoring-service }
spec:
  containers:
  - name: monitoring-service
    image: registry/monitoring-service:1.0
    ports: [ { containerPort: 8080 } ]
```

7.2 Learning Experiences

a. Knowledge Acquired

The internship deepened my understanding of cloud computing, distributed systems, and applied machine learning. Concepts that had been largely theoretical in coursework — such as message queues, microservices, and time-series forecasting — became concrete through direct application. The cloud-architecture knowledge connected closely with my Computer Networks and Operating Systems courses, while the machine-learning work built directly on my Artificial Intelligence elective.

b. Skills Learned

Several practical skills were developed during the internship. I became proficient in building REST APIs with Spring Boot, containerising applications with Docker, and deploying them on Kubernetes. Equally important were the softer skills: reading and understanding technical documentation, analysing ambiguous problems, breaking large tasks into manageable pieces, and collaborating effectively within an Agile team. A skill, in this sense, is the demonstrated ability to accomplish something reliably.

c. Observed Attitudes and Gained Values

Working alongside experienced engineers taught me values that no textbook conveys. I observed the importance of diligence — careful, persistent effort — in testing every change before release, and of accountability in owning the outcomes of one's work. I also came to appreciate intellectual humility: the willingness to admit uncertainty and ask questions, which I found was respected rather than penalised in a healthy engineering culture.

d. The Most Challenging Task

The most challenging task was integrating the Large Language Model so that its answers were both accurate and grounded in real infrastructure data. Early attempts produced fluent but sometimes incorrect responses — a phenomenon known as hallucination, where a model states something plausible but untrue. I overcame this by carefully designing the prompts to include a concise, factual summary of the current system state, and by constraining the kinds of questions the assistant would attempt to answer. This taught me that powerful tools require thoughtful guardrails.

7.3 SWOT Analysis

A SWOT analysis is a structured method for evaluating an organization by examining its internal Strengths and Weaknesses together with its external Opportunities and Threats. The analysis below reflects observations made about the host organization during the internship.

Table 7.1 SWOT Analysis Summary

Strengths – Deep expertise in cloud automation and DevOps. – Strong culture of innovation and learning. – Diverse client base across multiple industries. – Early adoption of AI technologies.	Weaknesses – Reliance on a limited number of large clients. – Talent retention is challenging in a competitive market. – AI offerings still maturing compared to core services.
Opportunities – Growing global demand for cloud and AI services. – Expansion into multi-cloud and edge computing. – Productising internal tools such as this assistant. – Partnerships with major cloud providers.	Threats – Intense competition from large system integrators. – Rapid technological change requiring constant upskilling. – Data-security and compliance risks in client systems. – Economic downturns reducing client IT budgets.

In summary, the organization is well positioned to capitalise on the rising demand for intelligent cloud services, provided it continues to invest in talent and diversify its client portfolio while managing the security risks inherent in operating other companies' infrastructure.